

Project Design Document

Group 11

Authors & Contributors:

Name	Email
Karine Davtyan	karineda@my.yorku.ca
Angelina De Santis	adsantis@my.yorku.ca
Yawei Dong	ydong77@my.yorku.ca
Manoj Ravichandran	manojr@my.yorku.ca
Samin Sharif	samin23@my.yorku.ca
Owen Zhu	owenz75@my.yorku.ca

Last updated: June 14, 2026

Status: Final

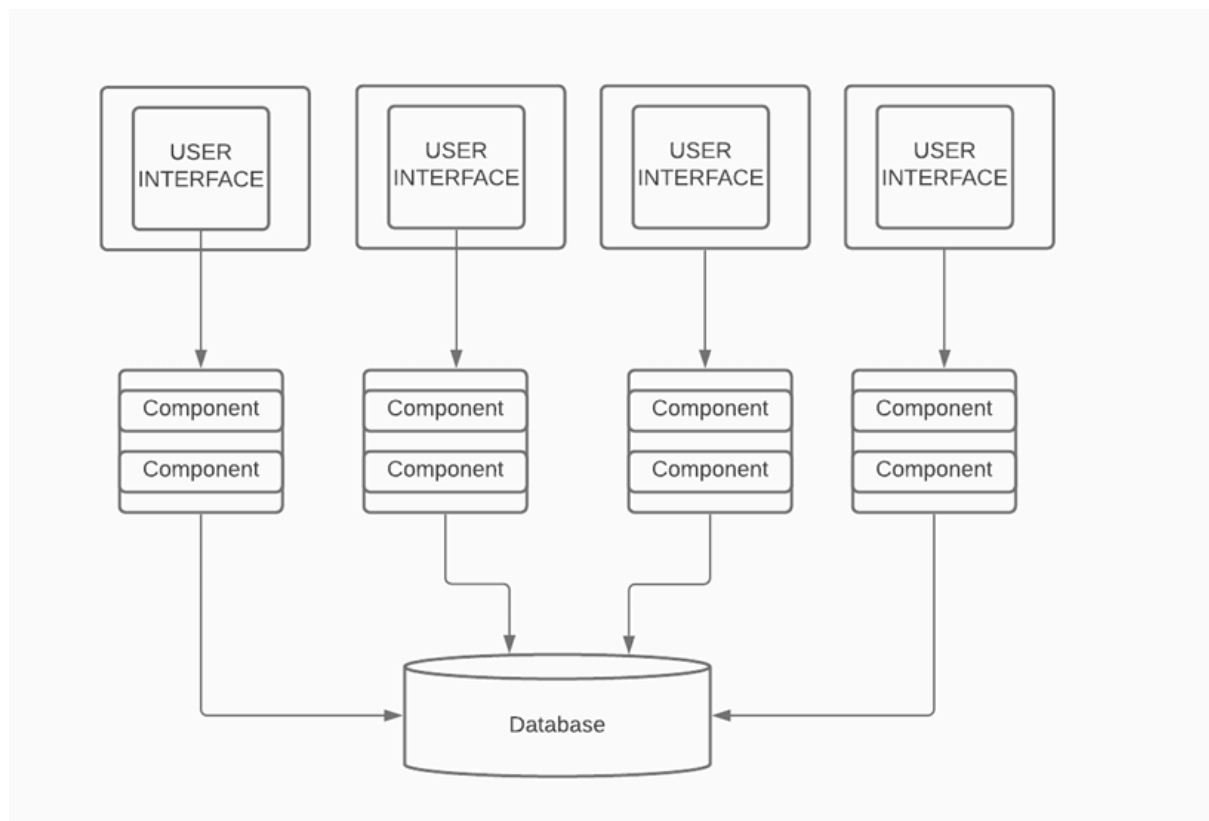
Table Of Contents

Purpose.....	3
Background Reading.....	4
Context.....	5
Detailed Design.....	6
Overall System Architecture.....	6
User Interaction With The System.....	7
How Software Components Interact.....	11
How Data Flows Through The System.....	12
Implementation Plan.....	16
Technologies We Intend To Use.....	16
Deploying the System.....	16
Source Code Management.....	17
Integrating Components.....	19
Testing the System.....	19
Project Planning.....	21
Team Organization.....	21
How Work Is Coordinated.....	21
Schedule.....	22
Appendix.....	23

Purpose

BookAtlas is a book tracking platform that allows users to engage with books they have read, are currently reading, or plan to read in the future. Our main concerns in developing this application are low costs, low latency and maintaining a separation of concerns across services.

The Service-Based Architecture style solves these problems due to its high ratings in cost, performance, and modularity as referenced in the slides. This design supports modularity through the creation of multiple service components, each with their own user interfaces, which in turn, supports testability and fault tolerance. Modularity also increases performance, because the frontend and backend are deployed separately, supporting independent scaling and smaller codebases. Furthermore, this design supports a team who prioritizes low costs because of the monolithic database, as introducing more databases is where costs usually spike.



("Architecture Styles 2", Slide 5, Gias Uddin)

Background Reading

The team referenced slides provided in the class about Service-Based Architecture and Martin Fowler's writings on microservices to understand how separating functionality into distinct services can improve maintainability and scalability. Additionally, we browsed through Google's recommended Python project structure (repository layout with separated modules) to guide how each service's codebase is organized. This informed the decision to structure BookAtlas around independent services for authentication, library management, book management, and discussions.

We also mentioned that in our project proposal, we were deciding between FastAPI and Express. After reading the official FastAPI documentation and researching quick API prototyping approaches, FastAPI was chosen for its automatic OpenAPI documentation generation and rapid API development capabilities. All of which simplify testing and long-term maintenance.

Our group initially built MongoDB schemas and database configurations in an early prototype, however following the professor's recommendation to adopt a data lake approach for its industry relevance and scalability patterns, that work has since been archived and the team is now exploring Databricks integration as the primary data storage solution.

Lastly, we looked into Redis and Meilisearch as potential tools for local book storage and fast, typo-tolerant searches. The main reason being to give users the ability to do elastic searches for books when they add or search so that in the frontend the query is instant and gratifying. After reading through their documentation, both seem promising but may be overkill for the project's current scope, this is still being evaluated as development progresses.

Context

For our design, a service-based architectural style handles multiple queries from multiple users to enhance user responsiveness. This allows a dedicated component to handle API queries to the backend for book cataloging, discussion forums, managing user libraries, and more. Dedicated components enable better accuracy and communication between components. Because there are dedicated components, each component can be tested separately, allowing test cases to be done in isolation. Another benefit to this design is the flexibility and scalability, allowing a 6-person team to develop the system's services in parallel at a relatively low cost for a successful deployment.

In a broader business sense, features can be implemented easily and evolve independently allowing for future adaptations and shorter delivery times. Team configuration can also scale based on the amount of features and scale of the actual database allowing for more or less parallel work to be done. Finally, because of isolated components, if a service fails, as long as the core system is still operational, the service causing issues can be identified easily and allow the website to continue functioning without interruption to the core service. For example; because discussion forums and personal libraries are separate services, even if the forums are closed, users can still access their library.

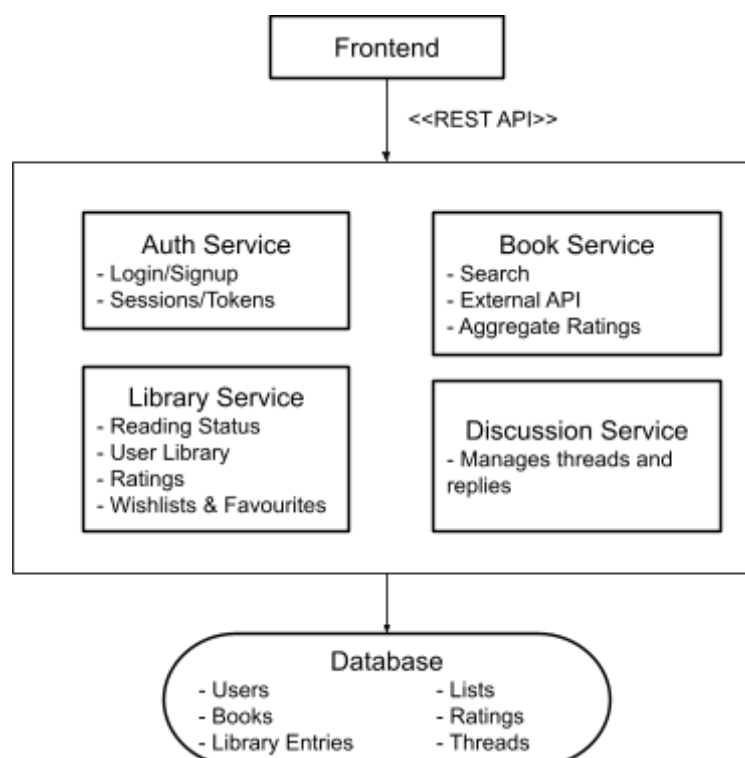
Detailed Design

Overall System Architecture

BookAtlas is a browser-based platform with a React frontend and a single backend application. The frontend handles page navigation, form validation, and user interactions. Whenever the frontend sends a request, it validates the inputs, then the backend service validates the request, and performs necessary operations and database queries before returning a response. After authentication, the backend creates a session or token and the frontend uses it to access protected features such as personal library management, profile management, and discussion forums.

The backend separates each domain, such as libraries and discussions, into internal services. The services communicate through shared identifiers (user_id, book_id, etc.) and internal interfaces/middlewares to keep the system modular and cohesive while avoiding the cost and complexity of microservices architecture style.

All data is persisted in a monolithic database, which stores core entities such as users, books, lists, ratings, threads, etc. This architecture provides a clean separation of concerns, and keeps the system maintainable and scalable to future updates.



User Interaction With The System

Users will interact with BookAtlas through a browser-based graphical user interface. The interface will provide navigation to the book catalog, personal library, wishlist/favourites lists, profiles, and forums.

1. Registration and Login

A visitor can create an account by entering a username, email address, and password. The frontend will perform basic validation, such as checking required fields and password length, before sending the registration request to the Authentication Service.

After registration or login, the backend will create an authenticated session or return an authentication token. The frontend will then allow the user to access protected features such as managing a personal library, rating books, and posting in forums.

Invalid login credentials or registration data will result in a clear error message being displayed to the user.

2. Searching and Viewing Books

A user (authenticated or unauthenticated) can search for books by title, author, genre, or ISBN. Search requests will be sent to the book service.

Search results are displayed as book cards containing information such as:

- Cover image
- Title
- Author
- Genre
- Average rating
- Reading status, when applicable

Selecting a book will open its details page, where the user can view its description, rating, reading status options, and related discussion.

3. Managing a Personal Library

An authenticated user can add a book to their personal library and assign one of the following reading statuses:

- Currently Reading
- Completed
- Dropped

The user can later update the reading status or remove the book from their personal library. The library page will allow books to be filtered by reading status, genre, and rating.

Each library entry belongs to one user and one book. A user cannot have duplicate library records for the same book.

4. Wishlist and Favourites

Authenticated users can add books to the Wishlist and Favourites lists independently of their reading status.

A book may be marked as both Currently Reading/Completed and a Favourite. Adding a book to one list will not automatically remove it from another list.

Wishlist and Favourites will be represented as predefined user lists. The data model will allow custom lists to be added in a future version without redesigning the core list system.

5. Rating Books

An authenticated user can assign a rating from one to five stars to a book. Each user can have only one active rating per book, but the rating may be updated or removed.

The global average rating will be calculated from all current user ratings. To avoid inconsistency, the average rating will be calculated using an aggregate database query instead of being stored as a separately maintained value.

6. Discussion Forums

BookAtlas will provide discussion forums associated with individual books and genres.

Users can create threads under the following categories:

- Spoilers
- Questions
- Theories

Each thread will contain:

- Title
- Main content
- Author
- Category
- Related book or genre
- Creation and modification timestamps

Authenticated users can reply to threads. Unauthenticated users may only view threads. Replies may support a hierarchical structure by referencing an optional parent reply.

Users may edit or delete only the threads and replies that they authored, unless they have an administrative role. Spoiler content will be clearly labelled and may be hidden until the user chooses to reveal it.

7. Public Profiles

Each user will have a public profile containing:

- Username
- Biography

- Profile picture
- Selected reading history
- Favourites
 - Ratings or rating history

Private information, such as email addresses and authentication credentials, will not appear on public profiles. Users will be able to edit only their own profile.

How Software Components Interact

The frontend will not access the database directly. All data operations will pass through the backend API.

The backend services will be logically separated by responsibility, but they will run within one backend application. Therefore, interactions between services will normally occur through internal application interfaces/middlewares and shared database identifiers rather than through separate network requests. This helps avoid messy point-to-point integration, which reduces testability and fault tolerance.

The services will interact in the following ways:

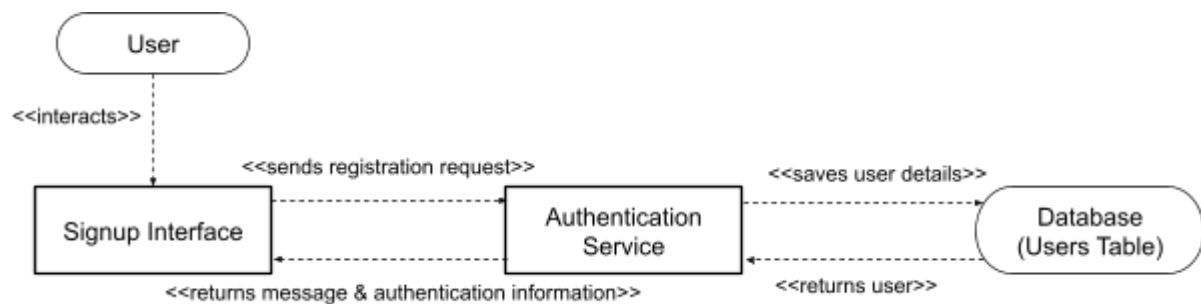
- The Authentication Service provides the authenticated user identity required by protected operations.
- The Library Service uses book identifiers associated with book records managed by the Book Service.
- The Discussion Service uses user identifiers to identify thread and post authors.
- The Discussion Service uses book and genre identifiers to associate discussions with the correct forum.
- The Book Service reads user rating records to calculate each book's average rating.
- The Profile functionality retrieves information from the user, such as their profile details, library, list, and ratings.

Services will not create duplicate copies of core entities. For example, the Library Service will store a `book_id` rather than copying the title, author, and description into the user's library record.

How Data Flows Through The System

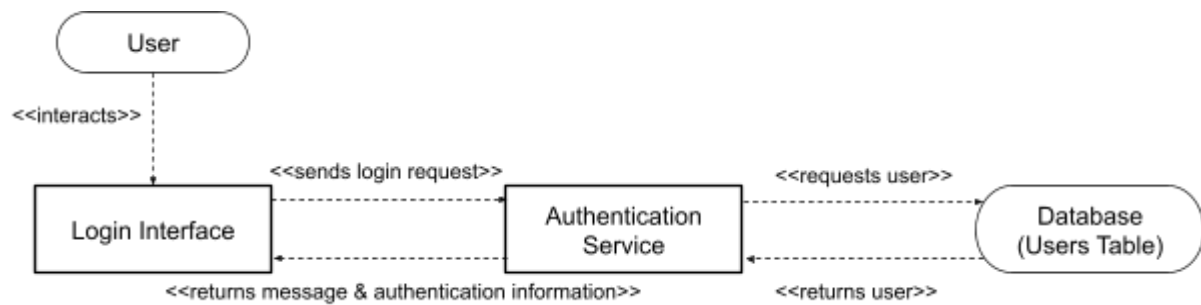
1. Registration Data Flow

- a. The user enters a username, email address, and password.
- b. The React frontend validates the required fields.
- c. The frontend sends a registration request to the Authentication Service.
- d. The Authentication Service validates the data and checks whether the username or email already exists.
- e. The password is hashed.
- f. A user record is inserted into the database.
- g. The backend returns a success response and authentication information.
- h. The frontend redirects the user to the dashboard.



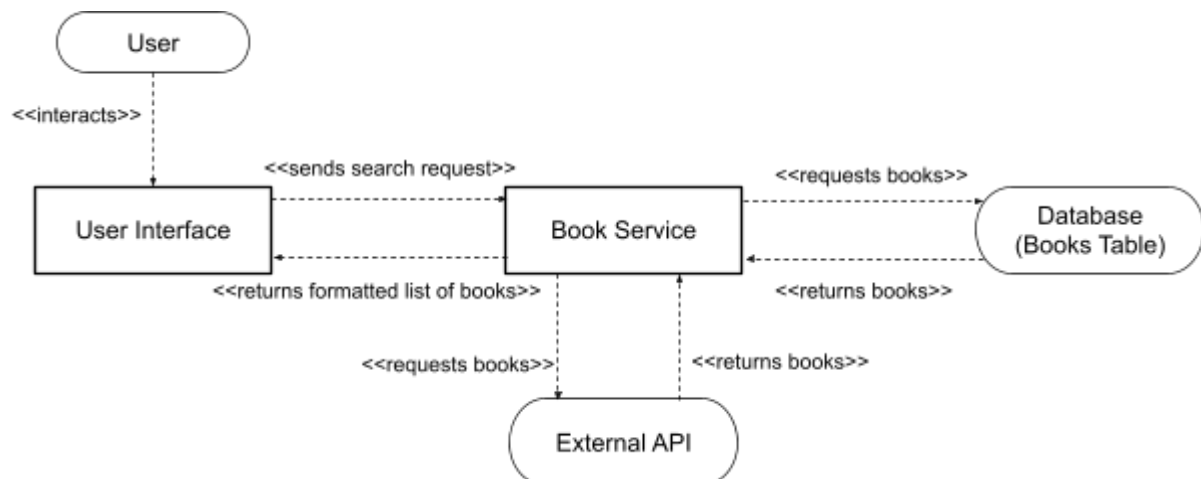
2. Login Data Flow

- a. The user enters an email address and password.
- b. The React frontend validates the required fields.
- c. The frontend sends a login request to the Authentication Service.
- d. The Authentication Service validates the data and checks whether the username or email already exists.
- e. The Authentication Service verifies the password.
- f. The service creates an authenticated session or token.
- g. The backend returns a success response and authentication information.
- h. The frontend redirects the user to the dashboard.



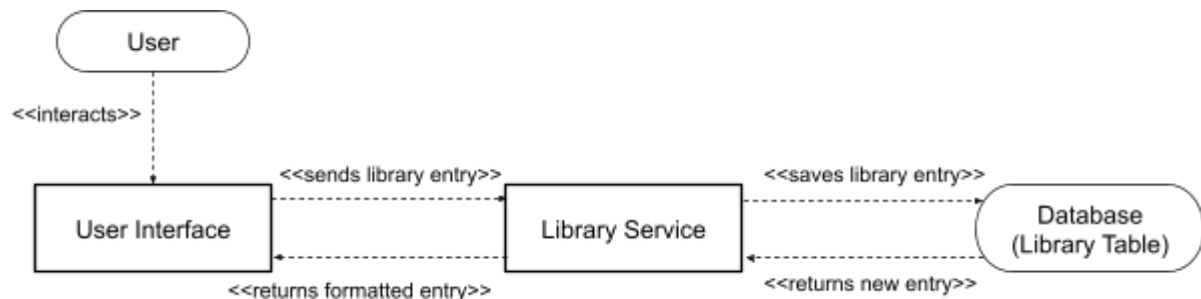
3. Book Search Data Flow

- The user enters a book title, author, genre, or ISBN.
- The frontend sends a search request to the Book Service.
- The Book Service queries the local database.
- If necessary, the Book Service sends a request to an external book API.
- External data is normalized into the BookAtlas data format.
- The backend returns the combined search results.
- The frontend displays the results as book cards.



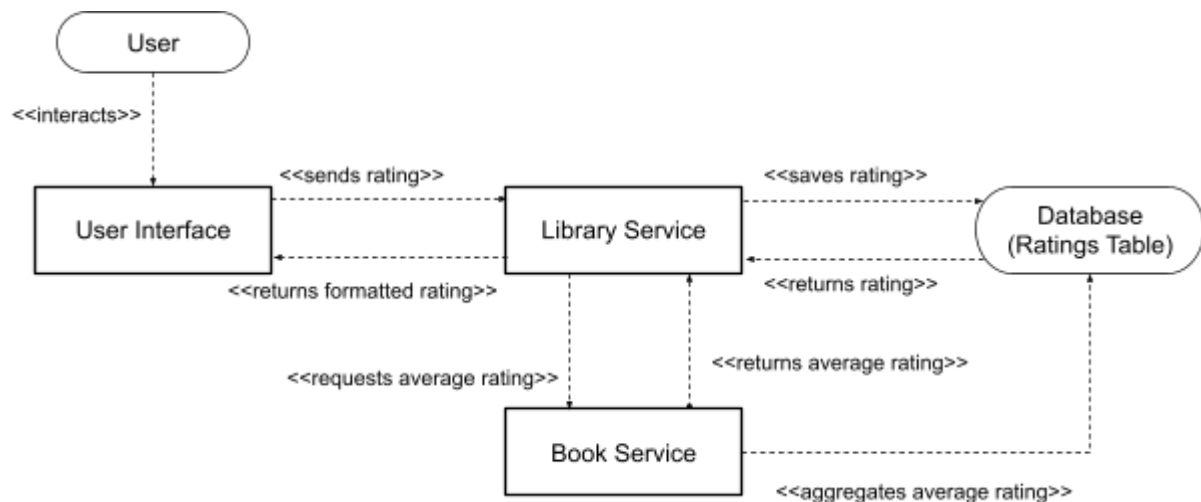
4. Add Book to Library Data Flow

- a. The authenticated user selects a book and reading status.
- b. The frontend sends the book ID and selected status to the Library Service.
- c. The backend validates the user's authentication.
- d. The Library Service verifies that the book exists.
- e. The service checks whether the user already has a library entry for the book.
- f. The entry is inserted or updated.
- g. The updated library record is returned.
- h. The frontend updates the displayed status.



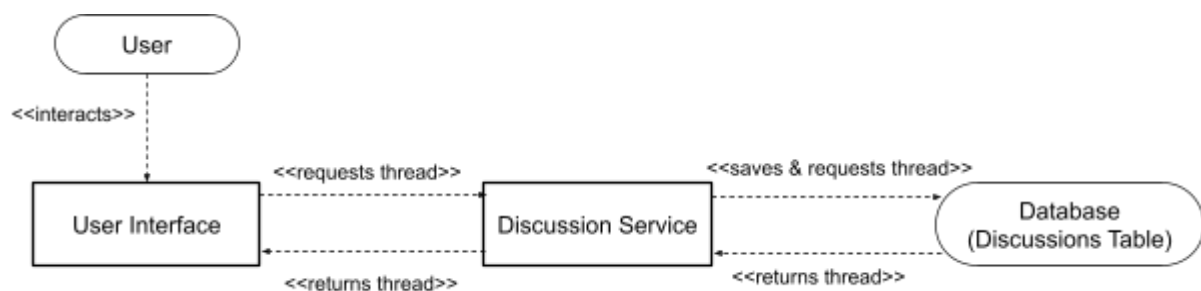
5. Rating Data Flow

- a. The user selects a rating from one to five stars.
- b. The frontend sends the rating and book ID to the Library Service.
- c. The service verifies the user and validates the rating.
- d. The rating record is inserted or updated.
- e. The Book Service calculates the new average from all ratings for the book.
- f. The updated user rating and average rating are returned.
- g. The frontend displays the new values.



6. Discussion Data Flow

- The user opens a book or genre forum.
- The frontend requests threads from the Discussion Service.
- The service queries threads associated with the requested forum.
- Only authenticated users can create a new thread or reply.
- The frontend sends the content and relevant identifiers to the Discussion Service.
- The service verifies authentication, validates the content, and confirms that the forum or thread exists.
- The content is stored in the database.
- The created thread or reply is returned and displayed.



Implementation Plan

Technologies We Intend To Use

- **Frontend:** For the frontend, we will use React because its component-based architecture makes the application easier to maintain and develop efficiently. Another factor in this decision is the team's existing experience with React. Additionally, we will use Shadcn and TailwindCSS to rapidly build and prototype the user interface.
- **Backend:** For the backend, we will use FastAPI for its rapid API development and built-in validation. We will attempt to use Redis to cache the results for all the book metadata and retrieve the information to send to the frontend as soon as possible for a good user experience. Although, this is not set in stone.
- **Database:** For the database side, we plan to use a data lake approach, closer to how Databricks handles data storage and processing. This would allow the system to store different types of data in one place and make it easier to run analytics or process large datasets later.

Deploying the System

For deployment, we plan to use Docker so the system can run inside containers. This helps make sure the application works consistently on different machines and makes deployment easier.

This means that a Dockerfile must be written for both the frontend and backend, allowing us to independently deploy and scale the frontend and backend services. Dockerfiles will be optimized for the smallest image size using multi-stage builds to increase build efficiency and scalability. Additionally, image size will be reduced by writing complete .dockerignore files so unnecessary files are not copied in the image.

Once deployed in Docker, the team will explore hosting platforms such as GCP, AWS, or Render to avoid hosting locally on our machines.

Source Code Management

The BookAtlas source code will be managed using Git and hosted in a shared GitHub repository. All team members will have access to the repository and will use Git for version control, collaboration, and tracking changes throughout the development process.

The project will use a monorepo structure, meaning that the frontend, backend, documentation, and configuration files will be stored in the same repository. The repository will be organized into separate directories so that each part of the system remains clearly separated.

A possible repository structure is:

- BookAtlas/
 - |- frontend/
 - |- src
 - |- components/
 - |- hooks/
 - |- lib/
 - App.jsx
 - |- backend/
 - |- auth/
 - |- database/
 - |- routers/
 - |- schemas/
 - |- services/
 - |- utils/
 - book_atlas.py
 - |- docs/
 - [README.md](#)
 - .gitignore
 - .env

The main branch will contain the stable and reviewed version of the project. Team members will not push changes directly to the main branch. Instead, each feature or bug fix will be developed on a separate branch created from the latest version of the shared code.

Branches will use descriptive names, such as:

- feature/user-login
- fix/rating-validation

After completing a task, the developer will push the feature branch to GitHub and create a pull request describing the changes and testing completed. At least one team member will review the code before it is merged. Team members will regularly pull the latest changes to reduce merge conflicts, and any conflicts will be resolved collaboratively by the members responsible for the affected components.

Commit messages will be short but descriptive so that the team can understand the history of the project, such as:

- Added user registration endpoint
- Created book search page

GitHub Issues or GitHub Projects will be used to track development tasks, bugs, and feature progress. Tasks may be organized into stages such as To Do, In Progress, In Review, and Completed. Each task will be assigned to the team member responsible for completing it.

Sensitive information, including database credentials, API keys, authentication secrets, and deployment configuration values, will not be committed to the repository. These values will be stored in environment variables. A .gitignore file will be used to exclude files such as .env, dependency folders, build outputs, temporary files, and local development settings.

The repository README will include instructions for installing dependencies, configuring environment variables, running the frontend and backend locally, executing tests, and contributing to the project. This will help all team members maintain a consistent development environment.

Integrating Components

Components will be integrated incrementally throughout development. The team will first define shared API endpoints and data formats so that frontend and backend members can work in parallel. The frontend may use mock data until the required backend endpoints are available, after which the components will be connected and tested feature by feature. Database and API changes will be documented and communicated to affected team members, and complete user flows will be tested before changes are merged into the main branch.

Testing the System

1. Unit Testing

Unit testing will be used to verify individual frontend and backend components independently. Backend unit tests will cover functions such as user registration validation, password verification, reading-status updates, rating validation, and discussion authorization. Frontend unit tests will verify components such as login forms, book cards, rating controls, and discussion input forms. Each team member will be responsible for writing and running unit tests for the components they develop before creating a pull request.

Tools:

- **Pytest:** Backend unit testing
- **Vitest:** Frontend JavaScript and TypeScript unit testing
- **React Testing Library:** React component testing

2. Integration Testing

Integration testing will verify that the backend services, API endpoints, and database work correctly together. These tests will cover operations such as registering an account, logging in, searching for a book, adding a book to a personal library, submitting a rating, and creating a discussion thread or reply. The tests will confirm that requests are processed correctly, database records are created or updated as expected, and appropriate responses are returned.

Tools:

- **Pytest:** Running backend integration tests
- **FastAPI TestClient:** Sending test requests to backend API endpoints
- **Postman:** Manually testing API requests, responses, and connected workflows

3. System Testing

System and end-to-end testing will verify that the complete BookAtlas application works correctly from the user's perspective. Tests will cover major workflows such as registering, logging in, searching for a book, adding it to the library, updating its reading status, submitting a rating, and participating in a discussion.

Tools:

- **Playwright:** Automated end-to-end testing of complete user workflows

4. Manual Testing

Manual testing will be used to evaluate aspects of the system that are difficult to verify through automated tests. Team members will test the application in supported browsers and on different screen sizes. Testing will focus on navigation, responsive layout, form usability, error messages, loading states, book displays, rating controls, and discussion features. The team will also test invalid inputs and unauthorized actions to confirm that the system responds clearly and correctly.

Tools:

- **Web Browsers:** Testing the application from a user's perspective
- **Browser Developer Tools:** Inspecting layouts, network requests, console errors, and responsive behaviour

Project Planning

Team Organization

- **Full-Stack**
 - Samin Sharif
 - Yawei Dong
- **Backend**
 - Karine Davtyan
 - Owen Zhu
- **Frontend**
 - Angelina De Santis
 - Manoj Ravichandran
- **Documentation**
 - Everyone

How Work Is Coordinated

We follow an Agile/Scrum type of setup focusing on frequent meetings, updated documentation, clear task assignments, and clear deadlines. At the beginning of each week, the team will meet to scope out planned features and assign everyone specific features that they are responsible for implementing. Our goal is to assign everyone 2-3 features per week, which will be clearly written in a separate document where developers can update their task status as they work. Once a team member has finished their assigned task, they will test it and work with developers in the other team or within their own team to successfully integrate it before moving onto their next task.

Additionally, once or twice a week we will coordinate a call where everyone can program together and test features in real time. This allows us to collaborate, help anyone that might need it, and increase efficiency since bugs, confusions, and integrations can be assessed on the spot.

Furthermore, the project report documents are worked on equally by each team member and the coordination for each call is done by Angelina.

Schedule

- **Goals for Check-In #1: Week of June 26th**
 - Authentication Service
 - Book Service
 - Library Service
 - Set up external book API
 - Seeding the database with books

 - Major Development Milestone:
 - Users can successfully create an account, login, search for books, and add books to their personal library.
 - The database is set up and seeded with some initial data to support initial user interactions with the system.

- **Goals for Check-In #2: Week of July 10th**
 - Wishlist & Favourites
 - Discussion Service
 - Data Analytics Dashboard

 - Major Development Milestone:
 - Users can successfully create wishlists and favourites lists, view threads, create threads, and reply to threads.
 - The homepage displays popular books, highly rated books, etc. using the data analytics dashboard.

- **Goals for Check-In #3: Week of July 24th**
 - Security & Testing
 - Notification System **[if time permits]**

 - Major Development Milestone:
 - Unauthenticated users cannot view protected routes or perform protected actions, such as replying to threads, or saving books.
 - Forms are protected against SQL and JavaScript injection.
 - Authentication information (session or token) is protected.
 - All failure paths have been thoroughly tested to ensure there are no failure points in the system

- Thorough test cases have been written for both the frontend and backend.
 - The frontend has a responsive design.
 - **[if time permits]** Users receive notifications when someone replies to their thread on the discussion forum.
- **Goals Until Presentation: Week of August 1st**
 - Polishing & Refactoring
 - Implementing Unfinished Features
 - Major Development Milestone:
 - Optimizing performance of backend functions and database queries.
 - Improving frontend UI styling.
 - Delivering a complete system that was tested end-to-end and ready for the presentation.
-

Appendix

- Fowler, M. and Lewis, J. (2014) *Microservices Guide*. Available at: <https://martinfowler.com/microservices/> (Accessed: 12 June 2026).
- FastAPI (n.d.) *FastAPI Documentation*. Available at: [FastAPI Documentation](#) (Accessed: 12 June 2026).
- Google APIs (n.d.) *GitHub Organization Page*. Available at: [Google APIs GitHub](#) (Accessed: 12 June 2026).